

Preliminary Design Review (PDR)

Proyecto: Plataforma de Reservas Distribuidas

Información del Documento y Equipo

- **Materia:** Sistemas Distribuidos
- **Profesor:** Jesus Ariel
- **Grupo / Equipo:** Pendiente
- **Fecha:** 17 de Agosto de 2026
- **Versión:** 1.0 - Definición de MVP 1

Integrantes:

1. **Juan Esteban Oliveros Duran** - Código: [Pendiente]
2. **Angie Valentina Flores** - Código: [Pendiente]
3. **Charith Nikool Chavarro** - Código: [Pendiente]
4. **Daniel Stiven Poveda** - Código: [Pendiente]

1. Visión Ejecutiva del Proyecto

El objetivo de este proyecto es diseñar e implementar un sistema de reservas de propiedades descentralizado. A diferencia de un monolito tradicional, este sistema se construirá desde el día uno asumiendo las realidades de la computación distribuida: latencia, fallos de red parciales e inconsistencia de estado. El enfoque principal será garantizar resiliencia, alta disponibilidad para operaciones de lectura y consistencia fuerte para transacciones críticas de negocio (reservas).

1.1. Alcance del MVP 1

Para el primer entregable, el sistema debe soportar el siguiente "Camino Feliz" y sus caminos de fallo:

1. El usuario visualiza propiedades disponibles.
2. El usuario selecciona fechas y solicita una reserva.
3. El sistema reserva temporalmente el inventario (fechas).
4. El sistema procesa el pago de forma asíncrona.
5. El sistema confirma la reserva o, en caso de fallo, ejecuta acciones de compensación para liberar el inventario.

2. Domain-Driven Design (DDD): Diseño Estratégico

El sistema se descompone en tres **Bounded Contexts** independientes, los cuales se implementarán como microservicios separados.

2.1. Bounded Context: Catálogo (Catalog Service)

- **Responsabilidad:** Gestión de la información de las propiedades (fotos, ubicación, amenidades) y motor de búsqueda.

- **Características Tácticas:** Altamente desnormalizado, optimizado para lectura rápida.
- **Dueño de Datos:** Almacenamiento Documental (ej. MongoDB) o motor de indexación (ej. Elasticsearch).

2.2. Bounded Context: Reservas (Booking Service)

- **Responsabilidad:** Gestión del inventario de tiempo. Determina si una propiedad está disponible en fechas específicas y maneja el ciclo de vida de la reserva (**PENDIENTE**, **CONFIRMADA**, **CANCELADA**).
- **Características Tácticas:** Protege la invariante de negocio más crítica: "No se puede reservar la misma propiedad en fechas superpuestas".
- **Dueño de Datos:** Base de Datos Relacional (ej. PostgreSQL) para asegurar propiedades ACID a nivel local.

2.3. Bounded Context: Pagos (Payment Service)

- **Responsabilidad:** Procesar cargos a tarjetas y mantener el registro de transacciones (ledger).
- **Características Tácticas:** Debe ser absolutamente idempotente para evitar cobros duplicados frente a reintentos de red.
- **Dueño de Datos:** Base de Datos Relacional.

3. Trade-offs de Consistencia (CAP y PACELC)

No aplicaremos una única política de consistencia, sino que la adaptaremos por operación:

Operación	Modelo de Consistencia	Justificación	Teorema CAP/PACELC
Búsqueda y visualización de catálogo	Eventual	Preferimos que el usuario siempre pueda ver casas, incluso si el dato tiene unos segundos de retraso (stale data).	AP (Availability over Consistency). Bajo PACELC: Else Latency (preferimos baja latencia).
Creación de Reserva (Bloqueo de fechas)	Fuerte (Linearizable)	El doble bloqueo de fechas genera conflictos de negocio graves. Requerimos bloqueo estricto en la base de datos de Booking.	CP (Consistency over Availability). Bajo PACELC: Else Consistency .

4. Patrones de Arquitectura y Resiliencia

4.1. Arquitectura Hexagonal (Puertos y Adaptadores)

Cada microservicio implementará estrictamente esta arquitectura para aislar el dominio de la infraestructura:

- **Core (Dominio + Casos de Uso):** Contiene la lógica de negocio pura. No conocerá HTTP, bases de datos o brokers de mensajería.
- **Puertos (Interfaces):** Definirán los contratos de entrada (Driving) y salida (Driven).
- **Adaptadores:** Implementarán los puertos (ej. Adaptador REST HTTP, Adaptador PostgreSQL, Adaptador Productor Kafka).

4.2. Flujo de Transacción Distribuida: El Patrón SAGA

Dado que no podemos usar transacciones ACID entre microservicios, el flujo de "Place Order" (Reserva) utilizará una Saga coreografiada basada en eventos:

1. **Booking Service:** Recibe petición de reserva. Crea el registro en estado **PENDIENTE** y bloquea las fechas temporalmente. Emite evento **ReservaCreada**.
2. **Payment Service:** Escucha **ReservaCreada**. Verifica la idempotencia (usando el ID de reserva). Intenta procesar el cargo.
 - *Si es exitoso:* Emite evento **PagoAprobado**. Booking Service lo escucha y cambia estado a **CONFIRMADO**.
 - *Si falla:* Emite evento **PagoRechazado**.
3. **Compensación:** Si Booking Service escucha **PagoRechazado**, ejecuta su acción compensatoria: libera las fechas bloqueadas y cambia el estado a **CANCELADA**.

4.3. Semántica de Entrega e Idempotencia

- **Comunicación:** Utilizaremos paso de mensajes asíncrono (ej. RabbitMQ/Kafka) con semántica de entrega **At-least-once** (al menos una vez).
- **Manejo de duplicados:** Asumimos que los mensajes llegarán duplicados. Todo consumidor (especialmente Payment Service) mantendrá una tabla o caché de **processed_event_ids**. Si un evento ya existe, se descarta silenciosamente, logrando el efecto de procesamiento *Exactly-once*.

5. Estrategia de Pruebas y Despliegue

5.1. Niveles de Pruebas

1. **Unitarias (Core):** Probaremos las entidades de dominio y casos de uso aislados. Rápido y barato.
2. **Integración (Adaptadores):** Usaremos **Testcontainers** para levantar instancias reales (pero efímeras) de bases de datos y brokers, asegurando que nuestras queries y mapeos ORM funcionan contra sistemas reales.
3. **Contratos (Consumer-Driven Contracts):** Garantizaremos que si Booking emite un evento, Payment sabe cómo parsearlo, evitando que el sistema se rompa por cambios de formato.